



PROJECT DOCUMENTATION

Equipify

Equipment Rental Marketplace Platform

Al-Zaytoonah University of Jordan

Department of Computer Science

Academic Year: 2025 – 2026

PROJECT TEAM

SUPERVISOR

Dr. Wael Zyadat

TEAM LEADER

Mahmoud Mohammad

202211206

TEAM MEMBER

Mohammad Jaber

202212437

TEAM MEMBER

Khaled Asha

202312446

01 — Project Overview

Equipify is a multi-platform equipment rental marketplace that connects equipment owners with renters in a trust-verified environment. Inspired by the Airbnb model, the platform enables individuals and businesses to list, discover, and rent equipment across diverse categories — from electronics and vehicles to fitness machines and real estate.

The system operates across three client platforms: a React-based web application served on port 5173, a Flutter mobile application for Android and iOS, and a RESTful ASP.NET Core 8.0 Web API backend on port 5000. All platforms communicate via HTTP/REST using JSON as the data interchange format, with multipart form-data for file uploads.

Platform Summary



Key Features

- **Bilingual Support:** Full Arabic (RTL) and English (LTR) interface across all platforms, with 214 translation keys on the web and 130+ on mobile.
- **Phone-Based Authentication:** Registration and login via phone number with OTP verification, JWT access tokens, and refresh token rotation.
- **Interactive Maps:** Leaflet-based map browsing on web, flutter_map on mobile, with geographic viewport filtering and click-to-place-pin for listing locations.
- **Dynamic Pricing:** Five rental units (hour, day, week, month, year) with corresponding cost fields and duration constraints.
- **Admin Moderation:** Listing approval workflow, user management, category management, and aggregate dashboard statistics.
- **Rating System:** 1–5 star ratings on completed rentals with owner average recalculation.

02 — Problem Statement

The equipment rental market faces several persistent challenges that reduce efficiency and trust between parties:

- **Fragmented Discovery:** Equipment owners and renters rely on word-of-mouth, social media groups, or scattered classified ads. There is no centralized, searchable platform that aggregates rental equipment by category, location, price, and availability.
- **Trust Deficit:** Without a verification mechanism, renters have no way to assess the reliability of an owner, and owners cannot verify the identity or intentions of renters. This leads to hesitation and failed transactions.
- **Manual Scheduling Conflicts:** Without an integrated booking system, overlapping rental requests are common, resulting in double-bookings and disputes.
- **Lack of Transparency:** Pricing, availability, and equipment condition are often unclear. Renters may encounter hidden fees or receive equipment that does not match the description.
- **No Accountability:** Without a rating or review system, there is no mechanism to reward reliable owners and renters or to flag problematic participants.
- **Limited Language Support:** Most existing platforms serve only English-speaking users, excluding Arabic-speaking populations who constitute a significant market segment.

Equipify addresses these challenges through a purpose-built, bilingual marketplace with verified identities, integrated booking with OTP confirmation, transparent pricing, and a community-driven rating system.

03 — Proposed Solution

Equipify provides a comprehensive, multi-platform solution that addresses each identified problem through the following architectural and functional decisions:

Centralized Marketplace

A unified platform where equipment owners can create detailed listings with images, pricing, location coordinates, and availability windows. Renters can browse, search, filter, and discover equipment through both grid and map views.

Verified Identity System

All user interactions require phone-based registration with OTP verification. This ensures that every participant on the platform has a verified phone number, establishing a baseline of accountability and traceability.

Integrated Booking Engine

The rental request workflow enforces date/time selection, automatic cost calculation, overlap detection against existing bookings, and OTP-verified confirmation. This eliminates double-bookings and provides a clear audit trail.

Admin Moderation Layer

All new listings enter a `Pending` state and require admin approval before becoming visible in the marketplace. Admins can approve, deactivate, or delete listings, and can block or unblock users, ensuring content quality and platform integrity.

Community Trust Mechanism

After a completed rental, the renter can submit a 1–5 star rating. The system enforces one rating per rental request and recalculates the owner's average rating in real time, creating a transparent reputation system.

Bilingual Architecture

The entire user interface — including navigation, forms, error messages, and notifications — supports Arabic (default) and English with full RTL layout support. Categories include a `NameAr` field for bilingual display.

04 — Objectives

The primary objectives of the Equipify project are:

- 1. Build a Cross-Platform Marketplace:** Develop a web application (React + TypeScript), a mobile application (Flutter + Dart), and a backend API (ASP.NET Core 8.0) that function as a cohesive equipment rental ecosystem.
- 2. Implement Secure Authentication:** Provide phone-based registration with OTP verification, JWT access tokens with configurable expiry (60 minutes for users, 120 minutes for admins), and refresh token rotation with 7-day expiry.
- 3. Enable Equipment Listing Management:** Allow authenticated users to create, edit, activate/deactivate, and delete equipment listings with image upload (maximum 6 images, 5MB each), map-based location selection, and dynamic pricing based on five rental units.
- 4. Provide Advanced Search and Discovery:** Implement text-based search across title, description, location, and owner name; category and price filtering; rental unit and duration filtering; geographic viewport filtering via map bounds; and paginated results (maximum 50 per page).

- 5. **Enforce a Booking Workflow:** Implement a rental request lifecycle with OTP verification, overlap detection, owner accept/reject, and post-rental rating.
- 6. **Support Administrative Oversight:** Provide an admin dashboard with aggregate statistics, listing moderation (approve/deactivate), user management (block/unblock/delete), and category CRUD.
- 7. **Deliver Bilingual UX:** Support Arabic and English with RTL/LTR layout switching, bilingual category names, and locale-aware date/number formatting.
- 8. **Ensure Security Best Practices:** Apply BCrypt password hashing, timing-attack mitigation, rate limiting, input validation, file upload security (extension whitelist, magic byte verification, GUID filenames), and security headers.

05 — Scope

In Scope

- User registration, login, and profile management via phone + OTP
- Equipment listing CRUD with image upload and map-based location
- Browse, search, and filter listings (text, category, price, unit, duration, map bounds)
- Rental request lifecycle: send OTP, create request, accept/reject, cancel
- Post-rental rating system (1–5 stars) with owner average recalculation
- Admin dashboard, listing moderation, user management, category management
- Full Arabic/English bilingual support with RTL layout
- Dark mode / light mode toggle on all platforms
- Web application (React 19 + TypeScript + Vite 8)
- Mobile application (Flutter 3.38 + Dart 3.9)
- Backend API (ASP.NET Core 8.0 with Clean Architecture)
- SQL Server 2022 database with Entity Framework Core 8.0
- Email notifications (configurable, disabled in development)

Out of Scope

- Payment processing and financial transactions
- Real-time chat or messaging between users
- Push notifications for mobile devices
- Equipment delivery or logistics tracking
- Multi-language support beyond Arabic and English
- Social media authentication (Google, Facebook, Apple)
- Native iOS/Android builds (Flutter cross-platform only)
- Load balancing or horizontal scaling infrastructure
- CI/CD pipeline configuration
- Automated integration or end-to-end testing suites

06 — Target Users

The system defines three primary roles, each with distinct access levels and capabilities:

Role	Access Level	Capabilities
Guest	Public (unauthenticated)	Browse listings, view listing details, view categories, search and filter, view map markers
User	Authenticated (JWT + role: User)	All Guest capabilities, plus: create/edit/delete own listings, send rental requests, accept/reject incoming requests, rate owners after completed rentals, edit profile, change password
Admin	Authenticated (JWT + role: Admin)	View dashboard statistics, approve/deactivate/delete any listing, block/unblock/delete users, create/edit/delete categories, view/delete all rental requests

User Personas

- **Equipment Owner (User):** An individual or business that owns equipment (tools, vehicles, electronics) and wants to generate income by renting it out. They create listings, manage availability, and respond to rental requests.
- **Renter (User):** A person who needs equipment temporarily — for a project, event, or short-term use. They search for equipment, compare options, and submit rental requests with OTP verification.
- **Platform Administrator (Admin):** A trusted operator who ensures marketplace quality by reviewing listings, managing user accounts, and maintaining categories.

07 — Functional Requirements

FR-01: User Registration and Authentication

- The system shall allow users to register using a phone number, first name, last name, email address, and password.
- The system shall send an OTP to the provided phone number before completing registration.
- The system shall hash passwords using BCrypt before storage.
- The system shall issue a JWT access token (60-minute expiry) and a refresh token (7-day expiry) upon successful login.
- The system shall support admin login via username and password with a separate 120-minute JWT (no refresh token).
- The system shall provide a forgot-password flow using OTP verification.

FR-02: Equipment Listing Management

- Authenticated users shall be able to create listings with title, description, category, location (address + latitude/longitude), rental unit, pricing, and images.
- The system shall accept five rental units: `hour` , `day` , `week` , `month` , `year` .
- The system shall accept up to 6 images per listing (5MB max, .jpg/.jpeg/.png/.gif/.webp).
- New listings shall enter `Pending` status and require admin approval before becoming visible.
- Owners shall be able to activate/deactivate their own listings.
- Editing a listing shall revert its status to `Pending` for re-review.

FR-03: Search and Discovery

- The system shall support text search across listing title, description, location address, and owner name (case-insensitive).
- The system shall support filtering by category, price range, rental unit, and duration range.
- The system shall support geographic viewport filtering via map bounds (west/south/east/north).
- The system shall return paginated results with a maximum of 50 items per page.

FR-04: Rental Request Lifecycle

- Authenticated users shall be able to send a rental request for a listing with date range and time range.
- The system shall require OTP verification before creating a rental request.
- The system shall detect and prevent overlapping rental requests for the same listing.
- The listing owner shall be able to accept or reject incoming requests.
- The renter shall be able to cancel their own request.

FR-05: Rating System

- After an accepted rental, the renter shall be able to submit a 1–5 star rating for the owner.
- The system shall enforce one rating per rental request (unique constraint).
- The system shall recalculate the owner's average rating upon each new rating submission.

FR-06: Admin Operations

- The admin shall be able to view aggregate dashboard statistics (total users, listings, active listings, pending listings, categories, requests, pending requests).
- The admin shall be able to approve or deactivate any listing.
- The admin shall be able to block/unblock or delete any user.
- The admin shall be able to create, update, and delete categories (with bilingual names).
- The admin shall be able to view and delete any rental request.

FR-07: Internationalization

- The system shall default to Arabic (RTL) and support switching to English (LTR).
- The web application shall provide 214 translation keys; the mobile application shall provide 130+ translation keys.
- The system shall persist the user's language preference in `localStorage` (web) or `SharedPreferences` (mobile).

08 — Non-Functional Requirements

NFR-01: Security

- Passwords shall be hashed using BCrypt with auto-generated salt.
- The system shall implement timing-attack mitigation by running BCrypt even for unknown phone numbers.
- JWT tokens shall use HMAC-SHA256 signing with a configurable secret key.
- Refresh tokens shall be stored as SHA-256 hashes with a unique index.
- The system shall enforce rate limiting: 10 requests/minute on auth endpoints, 5 requests/minute on OTP endpoints.
- File uploads shall be validated against extension whitelist, Content-Type, and magic bytes.
- The system shall set security headers: `X-Content-Type-Options: nosniff` , `X-Frame-Options: DENY` , `Referrer-Policy: no-referrer` .

NFR-02: Performance

- The web application shall be built with Vite for fast development and optimized production builds.
- All pages shall be lazy-loaded via `React.lazy()` and `Suspense` .
- The API shall use Entity Framework Core with compiled queries for hot paths.
- Images shall be served as static files from `wwwroot/uploads/` .

NFR-03: Reliability

- All business logic shall return `ServiceResult` instead of throwing exceptions.
- Global exception handling shall convert exceptions to RFC 7807 ProblemDetails responses.
- The database shall use foreign key constraints with appropriate delete behaviors (Cascade, Restrict).

NFR-04: Usability

- The interface shall support full Arabic/English bilingual display with RTL/LTR layout switching.
- The design shall follow a consistent visual language with the Inter font family (English) and Cairo (Arabic).
- The web shall use a glassmorphism-based design system; the mobile shall use Material 3 with custom theming.

NFR-05: Maintainability

- The backend shall follow Clean Architecture with four layers: Domain, Data, Service, Api.
- The system shall use the Repository + Unit of Work pattern for data access.
- All services shall be registered via Dependency Injection (Scoped for most, Singleton for OTP and AdminAuth).

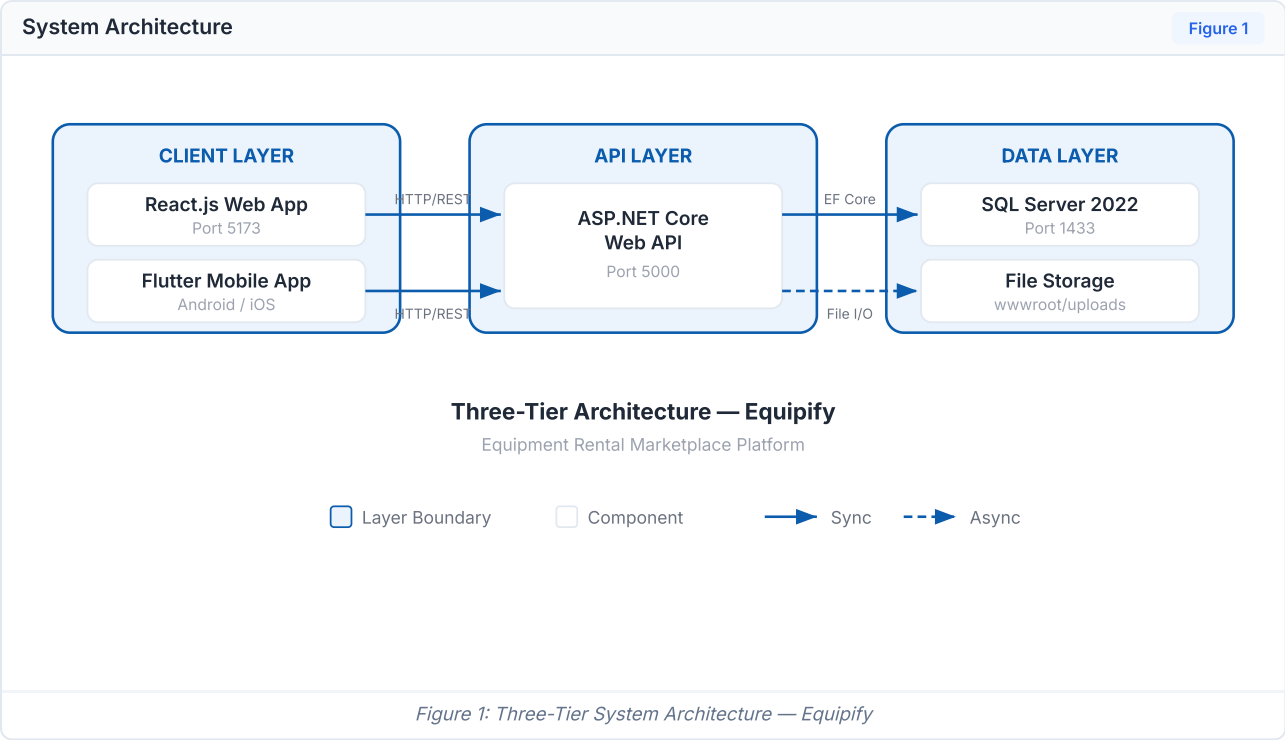
09 — System Modules

The Equipify system is organized into seven functional modules, each implemented across the backend, web, and mobile platforms:

Module	Description	Backend Services
Auth	Phone-based registration with OTP, JWT + Refresh tokens, admin login	UserService, OtpService, AdminAuthService, TokenService
Listings	CRUD with image upload, admin approval workflow, map search	ListingService, FileStorageService
Categories	Admin-managed with bilingual names (Name, NameAr)	CategoryService
Rental Requests	Booking flow with OTP verification, accept/reject, overlap detection	RentalRequestService, OtpService
Ratings	1–5 star ratings on completed rentals, owner average recalculation	RatingService
Admin Dashboard	Aggregate stats, content moderation, user/category management	DashboardService
i18n	Full Arabic/English support with RTL layout switching	N/A (client-side)

10 — System Architecture

Equipify follows a three-tier architecture with clear separation between presentation, business logic, and data storage layers:



Client Layer

The client layer consists of two independent applications that communicate with the backend via HTTP/REST:

- **React Web App** (port 5173): A single-page application built with React 19, TypeScript 6, and Vite 8. Uses Leaflet for interactive maps and React Context for state management.
- **Flutter Mobile App** (Android/iOS): A cross-platform mobile application built with Flutter 3.38 and Dart 3.9. Uses flutter_map for maps and the Provider package for state management.

API Layer

The backend is an ASP.NET Core 8.0 Web API running on Kestrel (port 5000). It exposes 48 RESTful endpoints organized across 6 controllers and implements JWT authentication, rate limiting, and global exception handling.

Data Layer

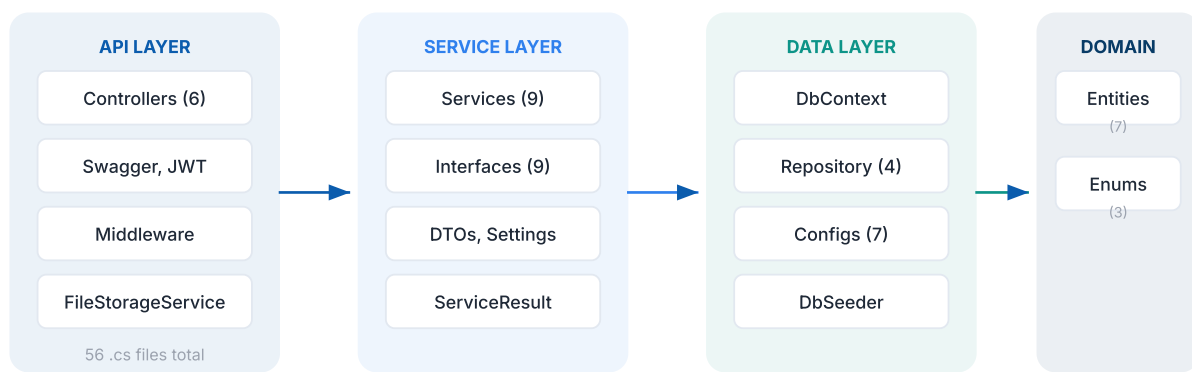
The data layer consists of SQL Server 2022 (running in Docker container `equipify-sql` on port 1433) and a local file system for uploaded images under `wwwroot/uploads/`.

11 — Backend Architecture

The backend follows Clean Architecture with four distinct layers, each with well-defined responsibilities and dependency directions:

Backend Layer Structure

Figure 2



Dependencies point inward → Domain has zero dependencies

Figure 2: Backend Clean Architecture — Dependency Graph

Project Breakdown

Project	Layer	Contents	Dependencies
Equipify.Api	API	6 Controllers, Swagger, JWT, Rate Limiting, Exception Middleware, FileStorageService	Service
Equipify.Service	Business Logic	9 Services (with interfaces), DTOs, Settings, ServiceResult pattern	Data
Equipify.Data	Data Access	DbContext, Generic Repository, Unit of Work, 7 EF Configurations, DbSeeder	Domain
Equipify.Domain	Domain	7 Entities (User, Listing, Category, ListingImage, RefreshToken, RentalRequest, UserRating), 3 Enums	None

Service Layer Details

Service	Lines	Responsibility	Registration
UserService	226	User CRUD, authentication, search, admin operations	Scoped
ListingService	282	Listing CRUD, browse with filters, moderation	Scoped
RentalRequestService	150	Booking lifecycle, OTP verification, overlap detection	Scoped
RatingService	80	Rating submission, average recalculation, reviews	Scoped
CategoryService	60	Category CRUD with listing existence check	Scoped
DashboardService	24	Aggregate statistics	Scoped
EmailService	74	SMTP email (disabled in dev, logs instead)	Scoped
OtpService	88	OTP generation, verification, in-memory store	Singleton
AdminAuthService	26	Admin credential validation	Singleton

Middleware Pipeline

1. Serilog (console + rolling file)
2. Data Layer registration (SQL Server + DI)
3. Service Layer registration (all services + settings)
4. JWT Authentication (HMAC-SHA256, 30s clock skew)
5. CORS (localhost:5173)
6. Rate Limiting (auth: 10/min, otp: 5/min)
7. Controllers (JSON enum converter, ignore nulls)

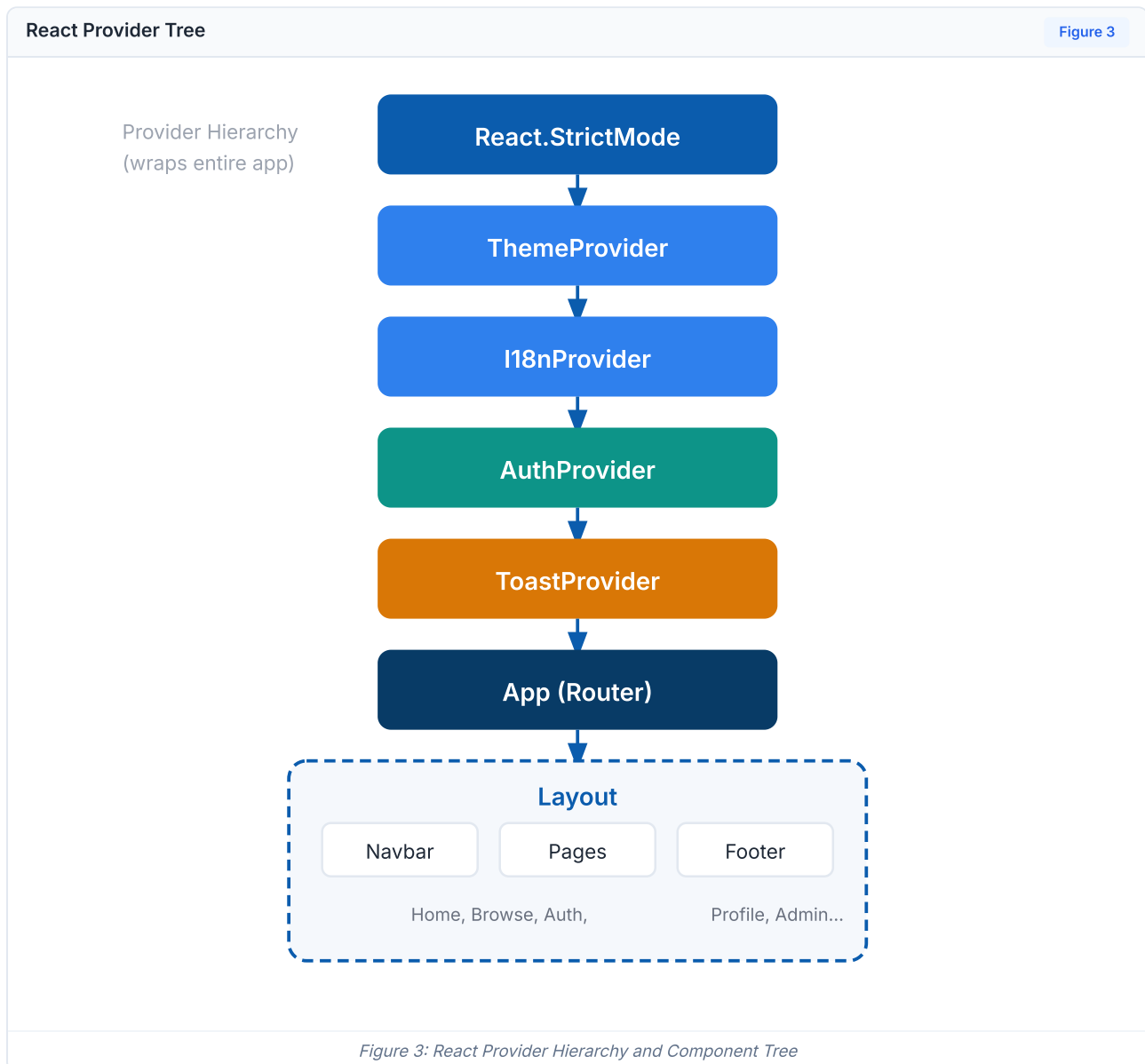
8. Swagger (always enabled, JWT Bearer support)
9. Database Seed (migrations + seed data on startup)
10. Security Headers (nosniff, DENY frame, no-referrer)
11. Static Files (wwwroot)

12 — React Architecture

Technology Stack

Technology	Version
React	19.2.8
TypeScript	6.0.2
Vite	8.2.0
react-router-dom	7.18.2
Leaflet / react-leaflet	1.9.4 / 5.0.0
eslint	1.75.0

Provider Hierarchy



State Management

Context	State
AuthContext	user, isAdmin, ready, login(), adminLogin(), register(), logout()
I18nContext	lang, dir, setLang(), toggle(), t()
ThemeContext	theme, toggle()
ToastContext	toast(message, kind?)

Routes

Path	Component	Auth
/	Home	Public
/browse	Browse	Public
/listings/:id	ListingDetails	Public
/login	Login	Public
/register	Register	Public
/profile	Profile	User
/my-listings	MyListings	User
/my-listings/new	ListingForm	User
/my-listings/:id/edit	ListingForm	User
/requests	MyRequests	User
/incoming	IncomingRequests	User
/faq	FAQ	Public
/terms	Terms	Public
/admin	AdminLayout	Admin
/admin/listings	AdminListings	Admin
/admin/users	AdminUsers	Admin
/admin/categories	AdminCategories	Admin
/admin/requests	AdminRequests	Admin

All pages are lazy-loaded via `React.lazy()` and `Suspense`.

API Client

The API client (`src/api/client.ts`) provides a fetch-based wrapper that auto-attaches JWT Bearer tokens, implements single-flight refresh on 401 responses, and supports both JSON and FormData payloads.

13 — Flutter Architecture

Technology Stack

Technology	Version
Flutter	3.38.5
Dart SDK	^3.9.0
provider	6.1.2
dio	5.7.0
flutter_map	7.0.2
shared_preferences	2.3.3

Technology	Version
image_picker	1.1.2
cached_network_image	3.4.1

State Management

The Flutter application uses the Provider package with ChangeNotifier. Three providers manage application state:

`AuthProvider` , `ThemeProvider` , and `LocaleProvider` .

Core Layer

- **ApiClient** (Singleton): Dio-based HTTP client with auto-attach Bearer token, refresh-token rotation on 401 via Completer (single-flight), and multipart form support.
- **Constants**: API URL configuration — `http://10.0.2.2:5000` (Android emulator), `http://localhost:5000` (iOS/desktop). Overridable via `--dart-define=API_URL=...` .
- **i18n**: Custom `S` class with map-based dictionary, 130+ keys, Arabic (default) + English.
- **Theme**: Material 3 with `ColorScheme.fromSeed` using accent `#2563EB` , custom component themes.

Models (14 Classes)

User, AuthSession, Category, ListingSummary, Listing, Owner, Paged<T>, MapMarker, OtpSent, RentalRequest, Renter, DashboardStats, Review.

Services (5 Classes)

Service	Methods
AuthService	register, sendOtp, login, adminLogin, forgotPassword, resetPassword, restore, me, updateProfile, changePassword, logout
CategoryService	all()
ListingService	browse, mapMarkers, details, mine, create, update, setStatus, delete, userReviews, listingReviews
RequestService	sendOtp, create, mine, incoming, accept, reject, cancel, rate
AdminService	dashboard, users, toggleUserStatus, deleteUser, listings, approveListing, deactivateListing, deleteListing, createCategory, updateCategory, deleteCategory, requests

Screens (12 Screens)

RootShell (bottom nav with IndexedStack), HomeScreen, BrowseScreen, ListingDetailsScreen, ListingFormScreen, MyListingsScreen, RequestsScreen, ProfileScreen, AdminHomeScreen, LoginScreen, RegisterScreen.

14 — Database Design

The Equipify database is hosted on SQL Server 2022 (Docker container `equipify-sql`) and managed via Entity Framework Core 8.0. It consists of 7 tables with well-defined relationships and constraints.

Tables

Table	Primary Key	Description
Users	Id (INT, Identity)	Registered users with phone, email, password hash, status, and average rating
Categories	Id (INT, Identity)	Equipment categories with bilingual names (Name, NameAr)
Listings	Id (INT, Identity)	Equipment listings with pricing, location, and status
ListingImages	Id (INT, Identity)	Image references for listings
RefreshTokens	Id (INT, Identity)	JWT refresh tokens (SHA-256 hashed)

Table	Primary Key	Description
RentalRequests	Id (INT, Identity)	Booking requests with date/time ranges and OTP verification
UserRatings	Id (INT, Identity)	Rental ratings and reviews (1–5 stars)

Relationships

Relationship	Cardinality	Delete Behavior
User → Listings	1:N	CASCADE
User → RentalRequests	1:N	RESTRICT
User → RefreshTokens	1:N	CASCADE
User → UserRatings (as Renter)	1:N	RESTRICT
User → UserRatings (as Owner)	1:N	RESTRICT
Category → Listings	1:N	RESTRICT
Listing → ListingImages	1:N	CASCADE
Listing → RentalRequests	1:N	CASCADE
Listing → UserRatings	1:N	RESTRICT
RentalRequest → UserRating	1:0..1	CASCADE

Seed Data

The `DbSeeder` runs on application startup and populates the database with:

- **9 Categories:** Electronics, Computers, Services, Vehicles, Motorcycles, Video Games, Fitness Machines, Real Estate, Equipments — each with Arabic names
- **3 Users:** Mahmoud Sanazmi, Mohammad Ali, Khaled Ahmad — with BCrypt-hashed password "123456"
- **4 Active Listings:** Tilt Trailer, Canon EOS M50, Lenovo ThinkPad X1 Carbon, BMW M5 — with images

15 — ER Diagram

The Entity-Relationship diagram below shows all database tables, their columns, primary keys, foreign keys, and relationships. Crow's foot notation is used for cardinality.

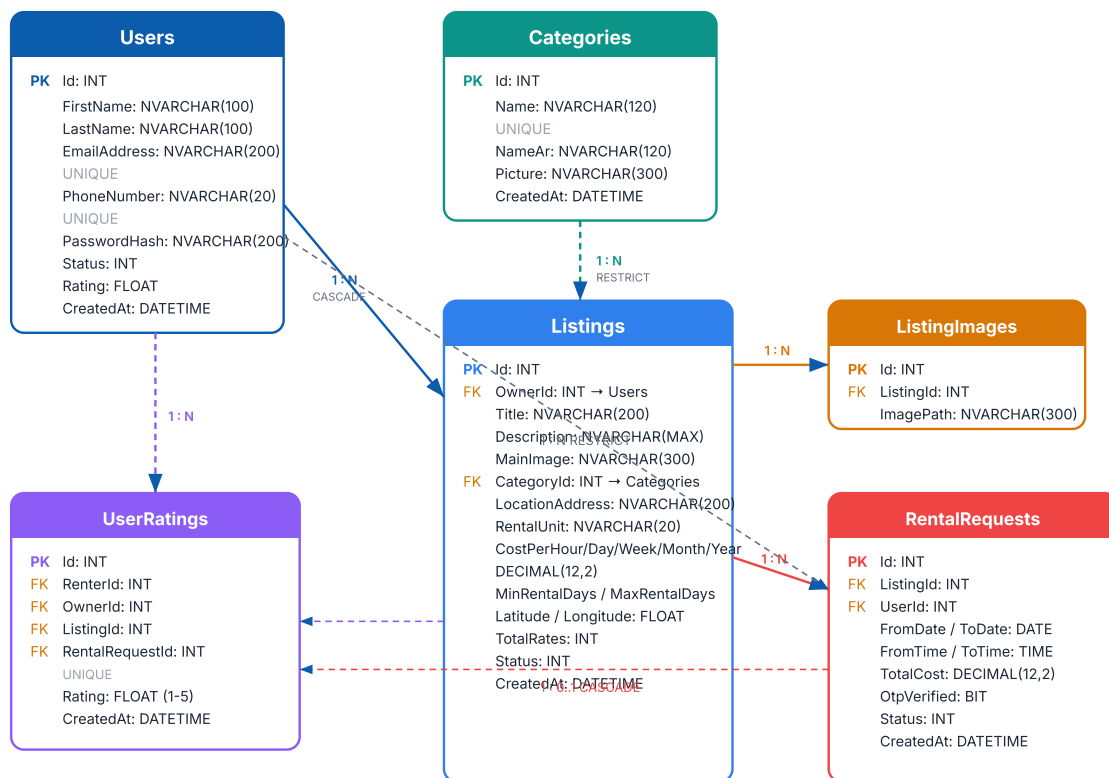


Figure 4: Entity-Relationship Diagram — Equipify Database

Key Constraints

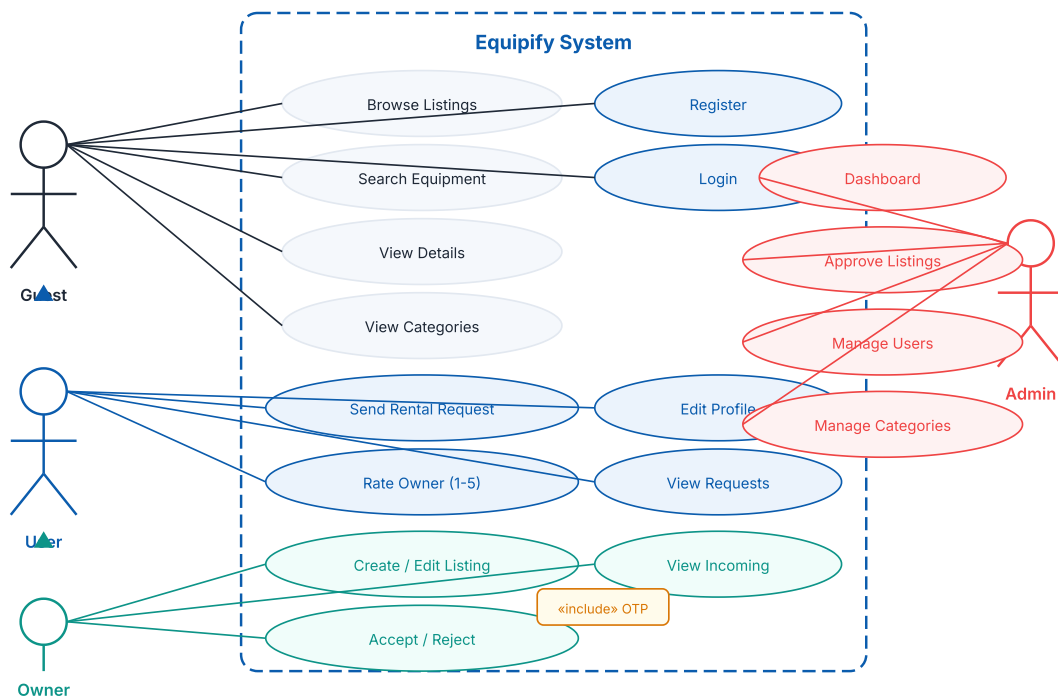
- Users:** Unique on `PhoneNumber`, Unique on `EmailAddress`
- RefreshTokens:** Unique on `TokenHash`
- UserRatings:** Unique on `RentalRequestId` (one rating per rental)
- Categories:** Unique on `Name`
- Listings:** `Precision(12,2)` on `CostPerHour`, `CostPerDay`, `CostPerWeek`, `CostPerMonth`, `CostPerYear`

Enum Values in Database

Column	Table	Values
Status	Users	Active (0), Blocked (1)
Status	Listings	Pending (0), Active (1), Inactive (2)
Status	RentalRequests	Pending (0), Accepted (1), Rejected (2)

16 — Use Case Diagram

The Use Case diagram identifies three actors — Guest, User (including Owner), and Admin — and their respective interactions with the system.



○ Actor ○ Use Case △ Generalization — Association ■ «include»

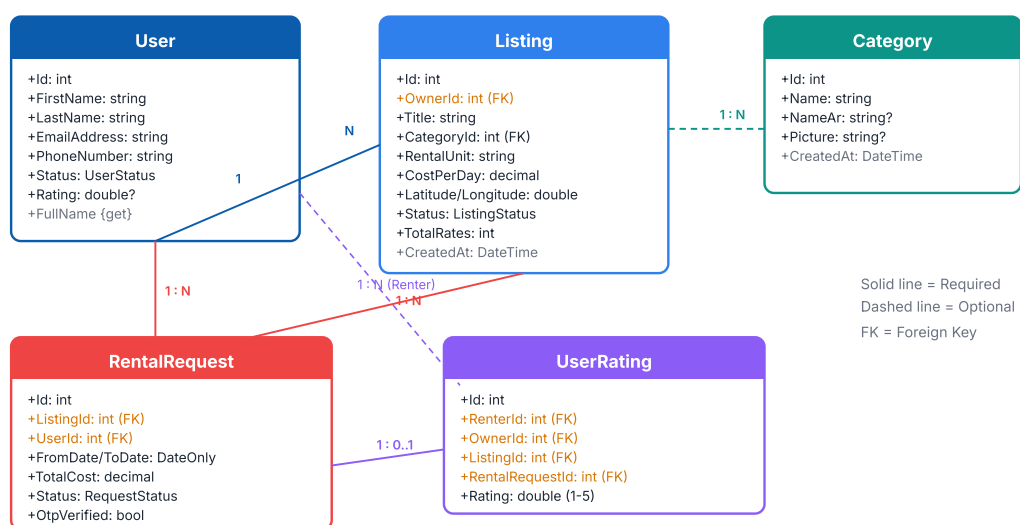
Figure 5: Use Case Diagram — Equipify

17 — Class Diagram

The Class diagram shows the seven domain entity classes, their fields, computed properties, navigation properties, and relationships.

Class Diagram — Domain Entities

Figure 6

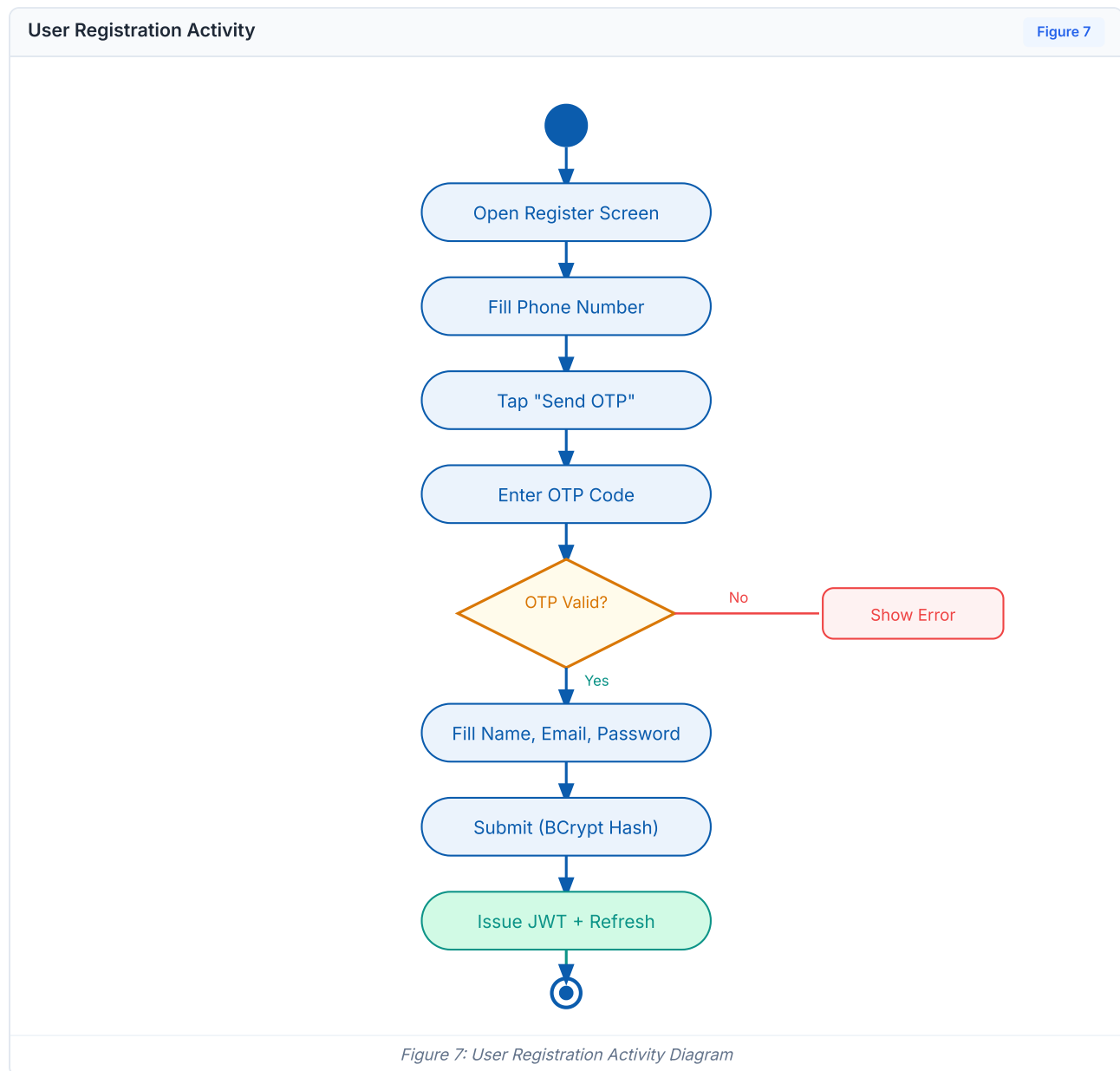


Solid line = Required
Dashed line = Optional
FK = Foreign Key

Figure 6: Class Diagram — Domain Entities and Relationships

18 — Activity Diagrams

18a: User Registration Flow



18b: Create Listing Flow

Create Listing Activity

Figure 8

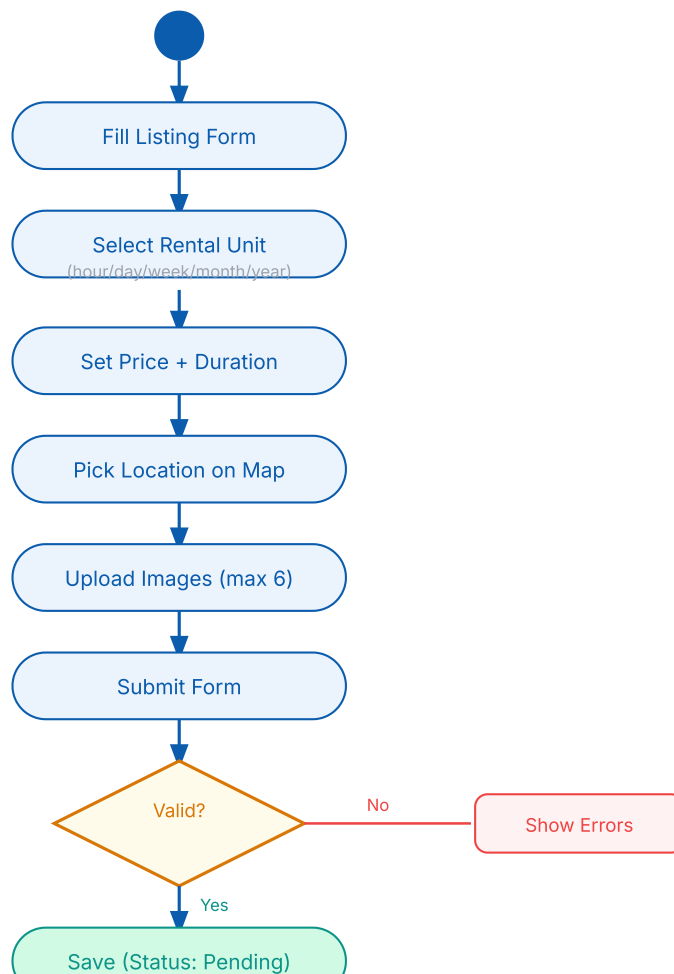
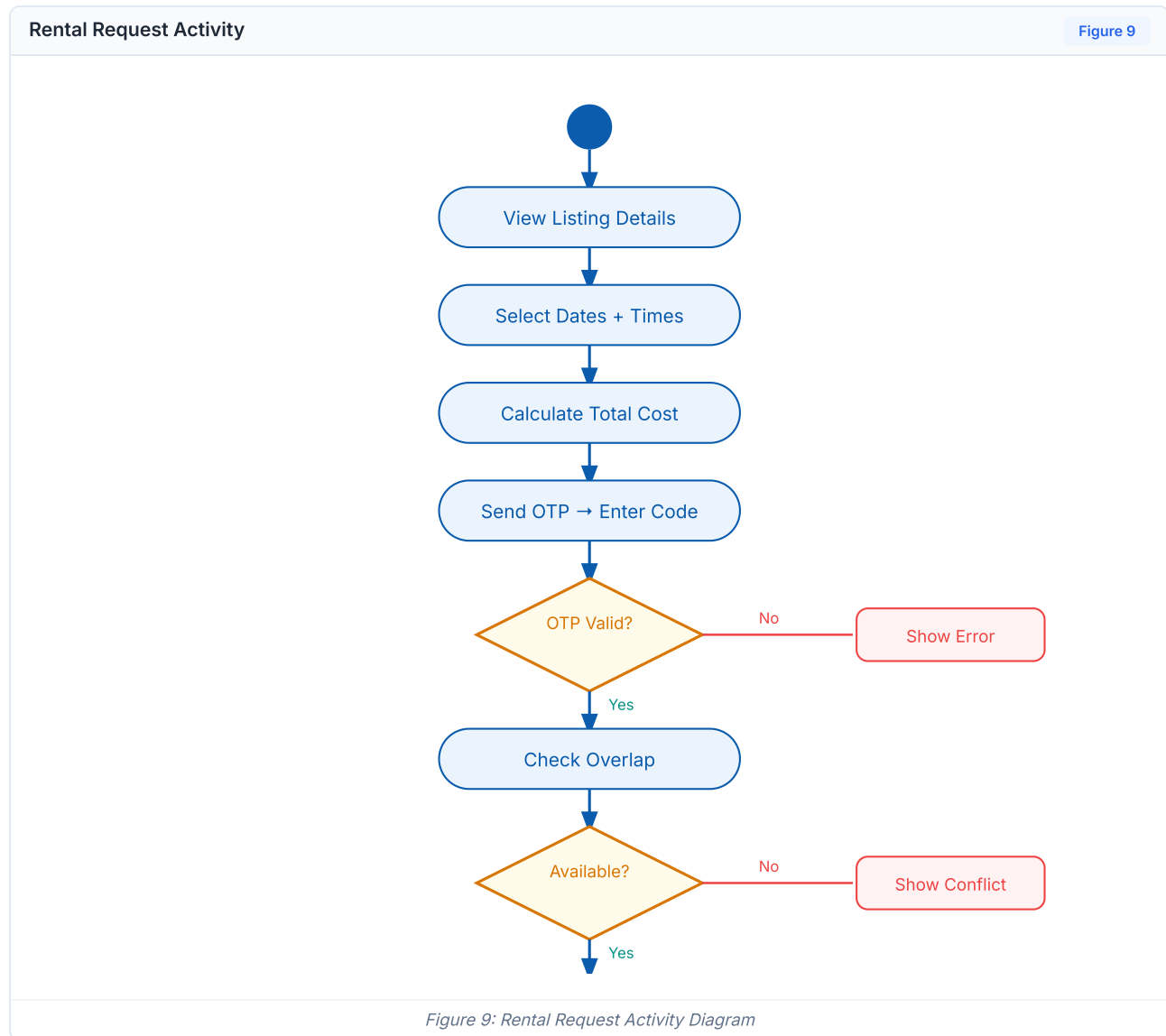


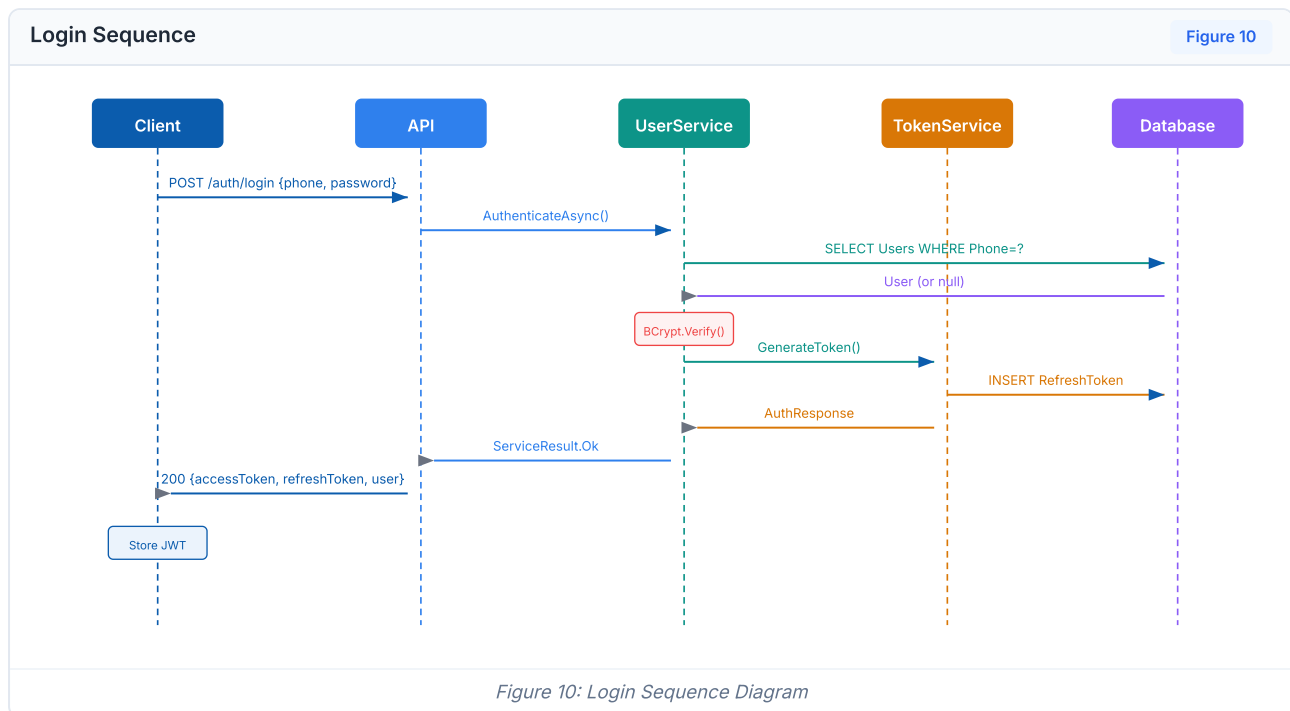
Figure 8: Create Listing Activity Diagram

18c: Rental Request Flow

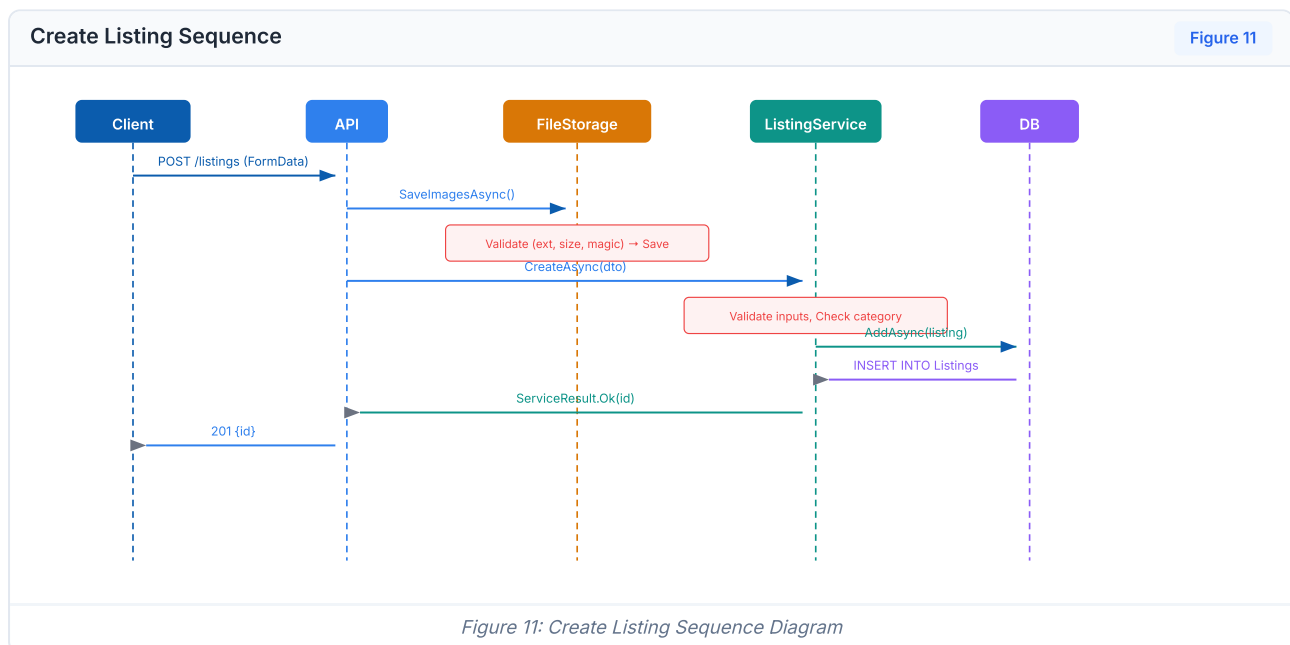


19 — Sequence Diagrams

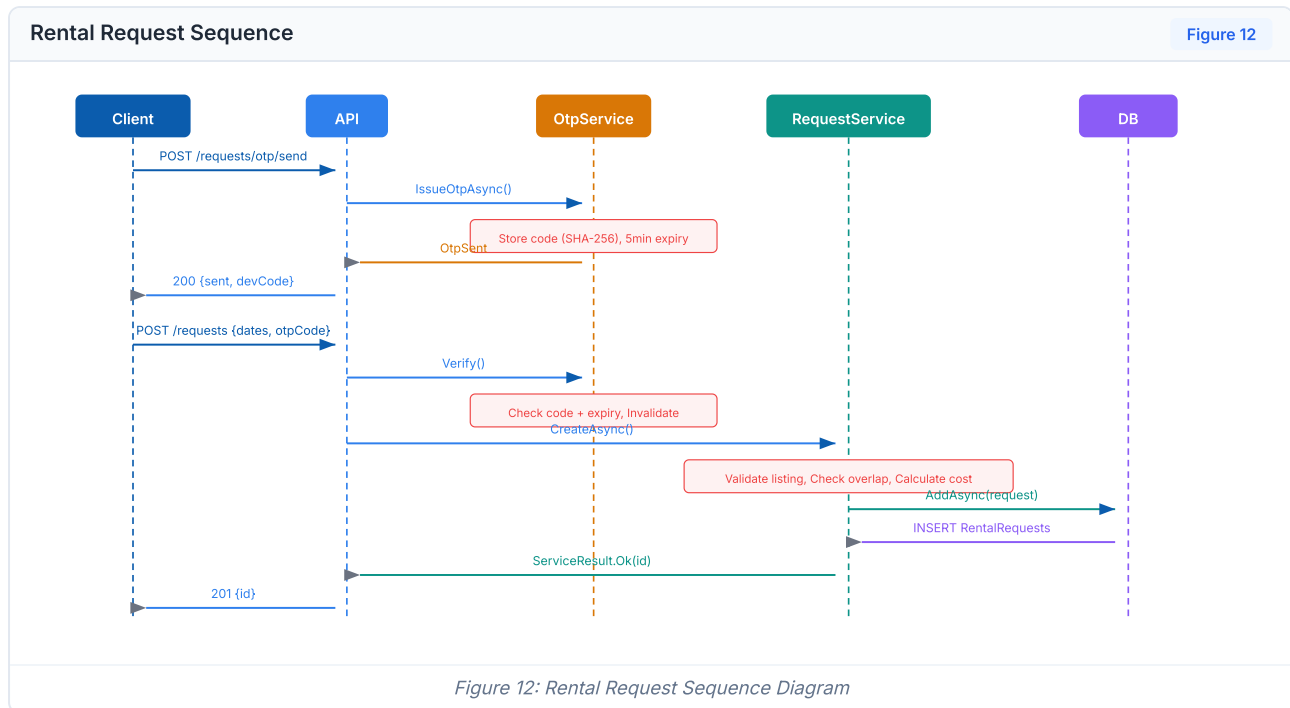
19a: Login Sequence



19b: Create Listing Sequence



19c: Rental Request Sequence



20 — Authentication Flow

The authentication system implements a multi-step flow with JWT access tokens, refresh token rotation, and OTP verification for sensitive operations.

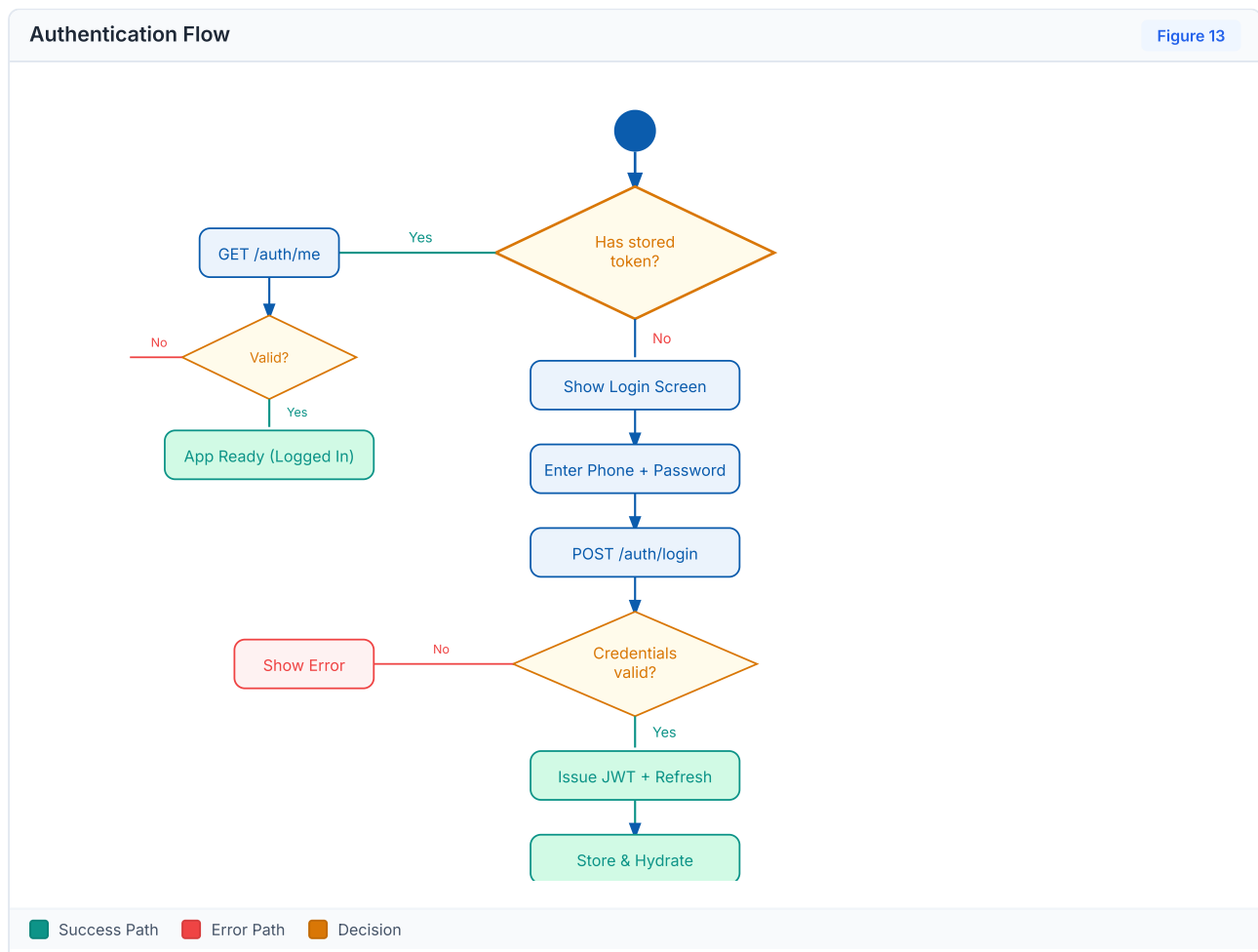


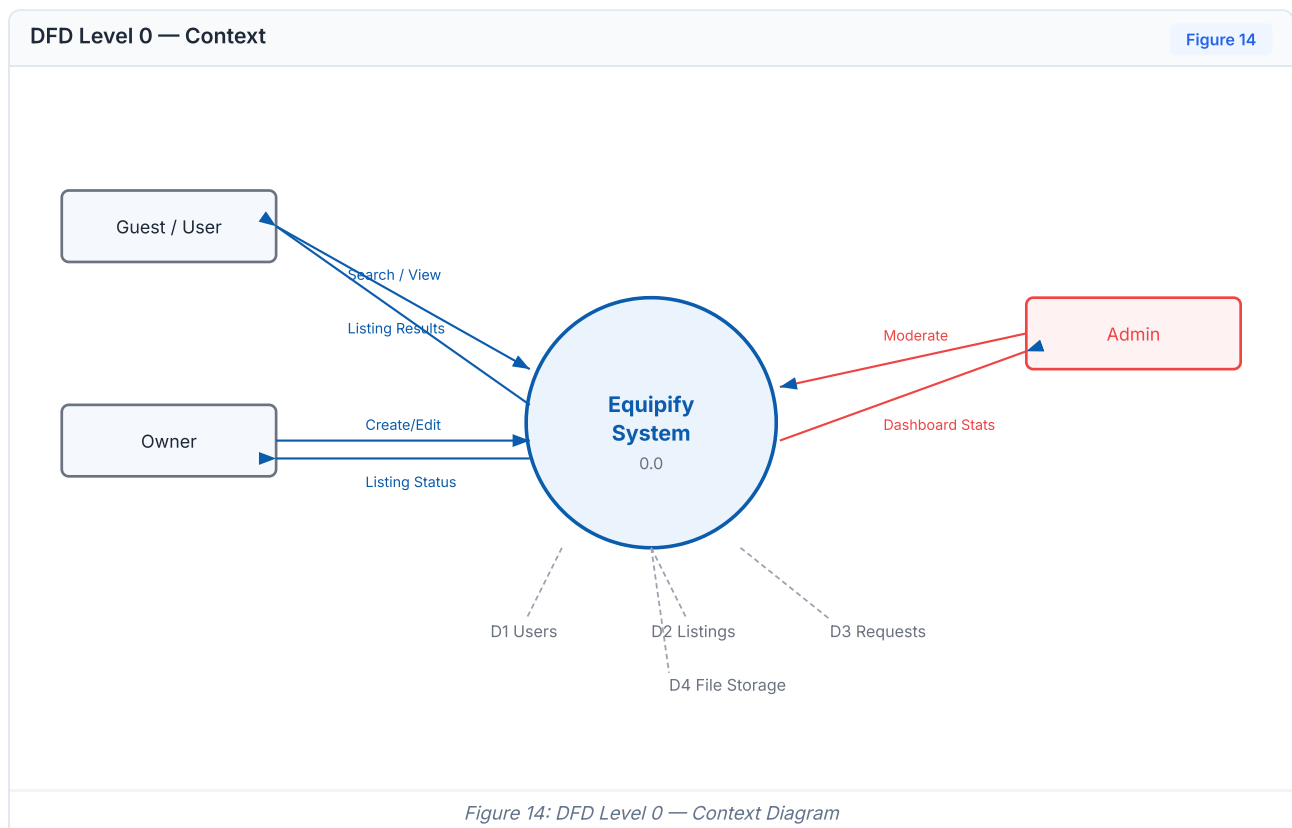
Figure 13: Authentication Flow Diagram

Security Properties

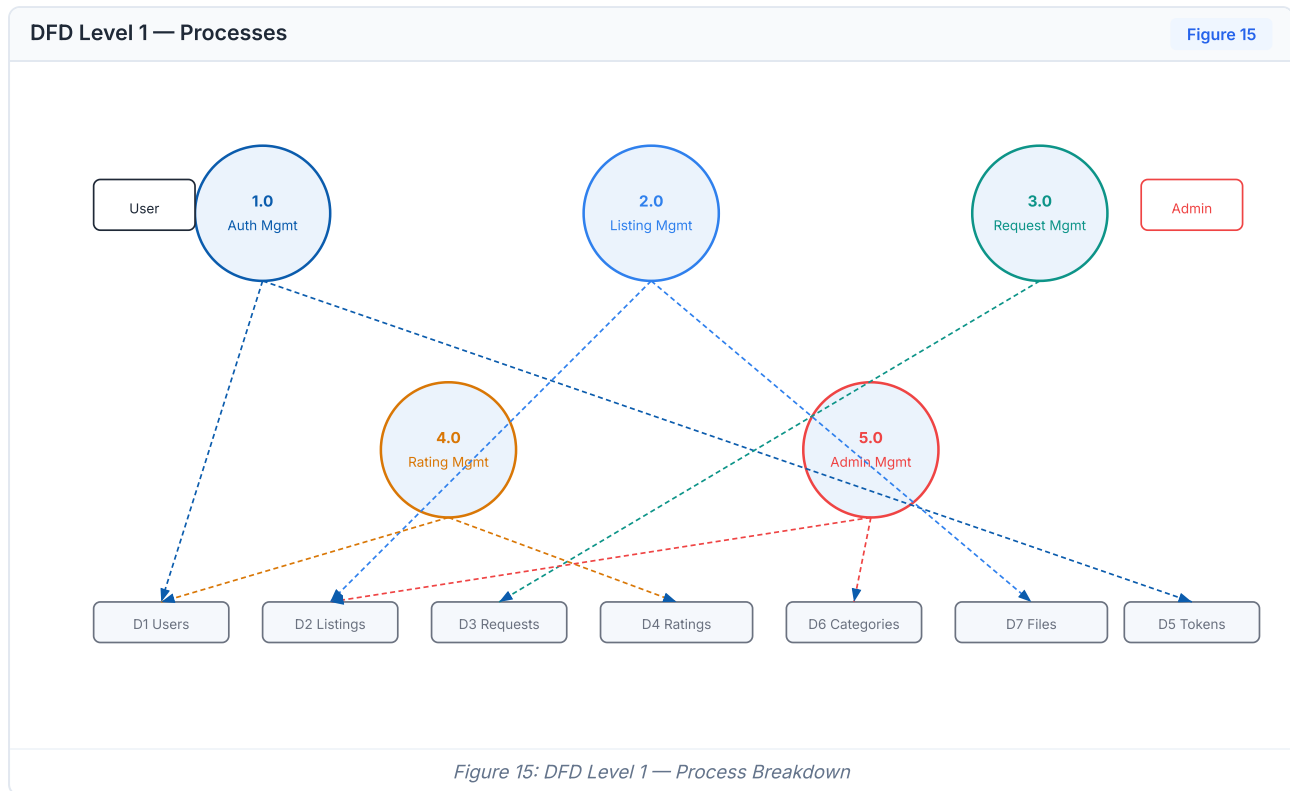
- **BCrypt Timing-Attack Mitigation:** BCrypt always runs even for unknown phone numbers, preventing timing-based user enumeration.
- **Refresh Token Rotation:** Old tokens are revoked on refresh, and new tokens are issued. SHA-256 hashes are stored in the database.
- **OTP Security:** Codes are stored as SHA-256 hashes with 5-minute expiry, 3 max attempts, and 60-second resend cooldown. Fixed code `0000` in development mode.
- **JWT Configuration:** HMAC-SHA256 signing, 60-minute user tokens, 120-minute admin tokens, 30-second clock skew tolerance.

21 — Data Flow Diagrams

DFD Level 0 (Context Diagram)

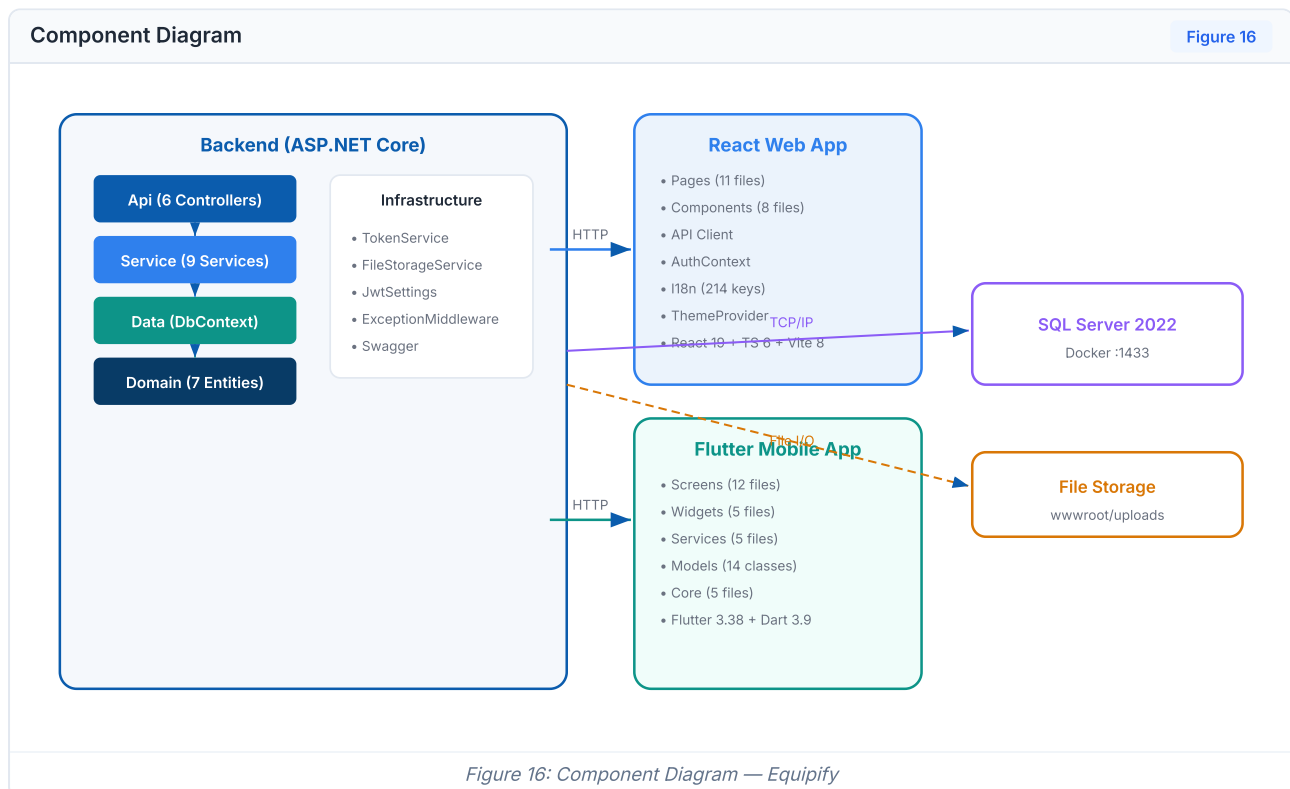


DFD Level 1



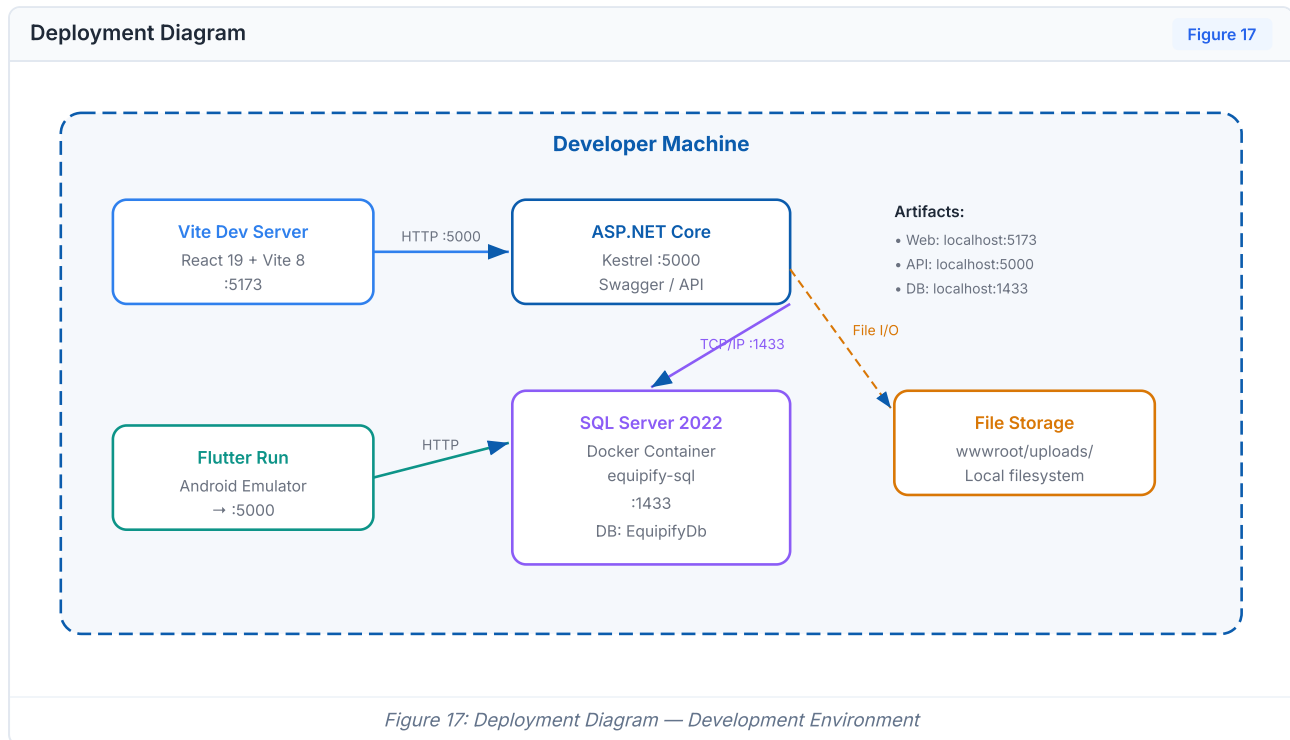
22 — Component Diagram

The Component diagram shows the software components and their dependencies across the three platforms.



23 — Deployment Diagram

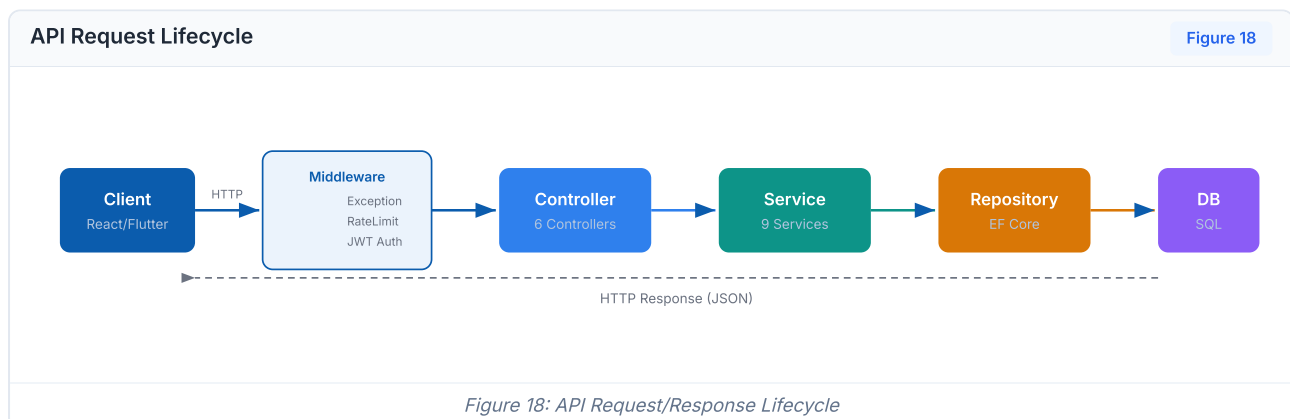
The Deployment diagram shows the development environment topology and communication paths between components.



24 — API Architecture

The Equipify API follows RESTful conventions with a consistent request/response pattern and a middleware pipeline that handles cross-cutting concerns.

Request/Response Flow



Middleware Pipeline

Every incoming request passes through the following middleware in order:

1. **ExceptionHandlerMiddleware:** Catches unhandled exceptions and converts them to RFC 7807 ProblemDetails responses.
2. **Rate Limiting:** Enforces per-IP throttling (10/min for auth, 5/min for OTP).
3. **JWT Authentication:** Validates Bearer tokens using HMAC-SHA256.
4. **CORS:** Allows requests from `localhost:5173`.

Endpoint Groups

Group	Base Path	Endpoints	Auth Required
Auth	/api/auth	13	Public / User
Admin	/api/admin	14	Admin
Listings	/api/listings	8	Public / User
Categories	/api/categories	2	Public
Requests	/api/requests	9	User
Ratings	/api	2	Public / User

ServiceResult Pattern

All business logic returns `ServiceResult` instead of throwing exceptions:

```
ServiceResult { bool Success; string? Error; }  
ServiceResult<T> : ServiceResult { T? Value; }
```


25 — API Endpoints

The following tables list all 48 API endpoints with their HTTP methods, authentication requirements, and rate limits.

Authentication Endpoints

#	Method	Endpoint	Body/Params	Rate Limit
1	POST	/api/auth/send-otp	{ phoneNumber }	10/min
2	GET	/api/auth/check-email	query: email	—
3	GET	/api/auth/check-phone	query: phone	—
4	POST	/api/auth/register	{ firstName, lastName, emailAddress, phoneNumber, password, otpCode }	10/min
5	POST	/api/auth/login	{ phoneNumber, password }	10/min
6	POST	/api/auth/admin-login	{ username, password }	10/min
7	POST	/api/auth/forgot-password	{ phoneNumber }	10/min
8	POST	/api/auth/reset-password	{ phoneNumber, otpCode, newPassword }	10/min
9	POST	/api/auth-refresh	{ refreshToken }	—
10	POST	/api/auth/logout	{ refreshToken }	—
11	GET	/api/auth/me	—	—
12	PUT	/api/auth/me	{ firstName, lastName, emailAddress, phoneNumber }	—
13	POST	/api/auth/change-password	{ currentPassword, newPassword }	—

Listings Endpoints

#	Method	Endpoint	Auth
14	GET	/api/listings (browse)	Public
15	GET	/api/listings/map	Public
16	GET	/api/listings/{id}	Public
17	GET	/api/listings/mine	User
18	POST	/api/listings (multipart)	User
19	PUT	/api/listings/{id} (multipart)	User
20	POST	/api/listings/{id}/status	User
21	DELETE	/api/listings/{id}	User

Categories Endpoints

#	Method	Endpoint	Auth
22	GET	/api/categories	Public
23	GET	/api/categories/{id}	Public

Rental Requests Endpoints

#	Method	Endpoint	Auth
24	POST	/api/requests/otp/send	User
25	POST	/api/requests	User

#	Method	Endpoint	Auth
26	GET	/api/requests/mine	User
27	GET	/api/requests/incoming	User
28	GET	/api/requests/for-listing/{listingId}	User
29	POST	/api/requests/{id}/accept	User
30	POST	/api/requests/{id}/reject	User
31	DELETE	/api/requests/{id}	User
32	POST	/api/requests/{id}/rating	User

Ratings Endpoints

#	Method	Endpoint	Auth
33	GET	/api/users/{id}/reviews	Public
34	GET	/api/listings/{id}/reviews	Public

Admin Endpoints

#	Method	Endpoint	Description
35	GET	/api/admin/dashboard	Aggregate stats
36	GET	/api/admin/users	All users
37	PUT	/api/admin/users/{id}	Update user
38	POST	/api/admin/users/{id}/toggle-status	Block/unblock
39	DELETE	/api/admin/users/{id}	Delete user
40	GET	/api/admin/listings	All listings
41	POST	/api/admin/listings/{id}/approve	Approve listing
42	POST	/api/admin/listings/{id}/deactivate	Deactivate listing
43	DELETE	/api/admin/listings/{id}	Delete listing
44	POST	/api/admin/categories	Create category
45	PUT	/api/admin/categories/{id}	Update category
46	DELETE	/api/admin/categories/{id}	Delete category
47	GET	/api/admin/requests	All requests
48	DELETE	/api/admin/requests/{id}	Delete request

26 — Security Architecture

The Equipify security architecture implements eight layered defenses to protect against common web application threats.

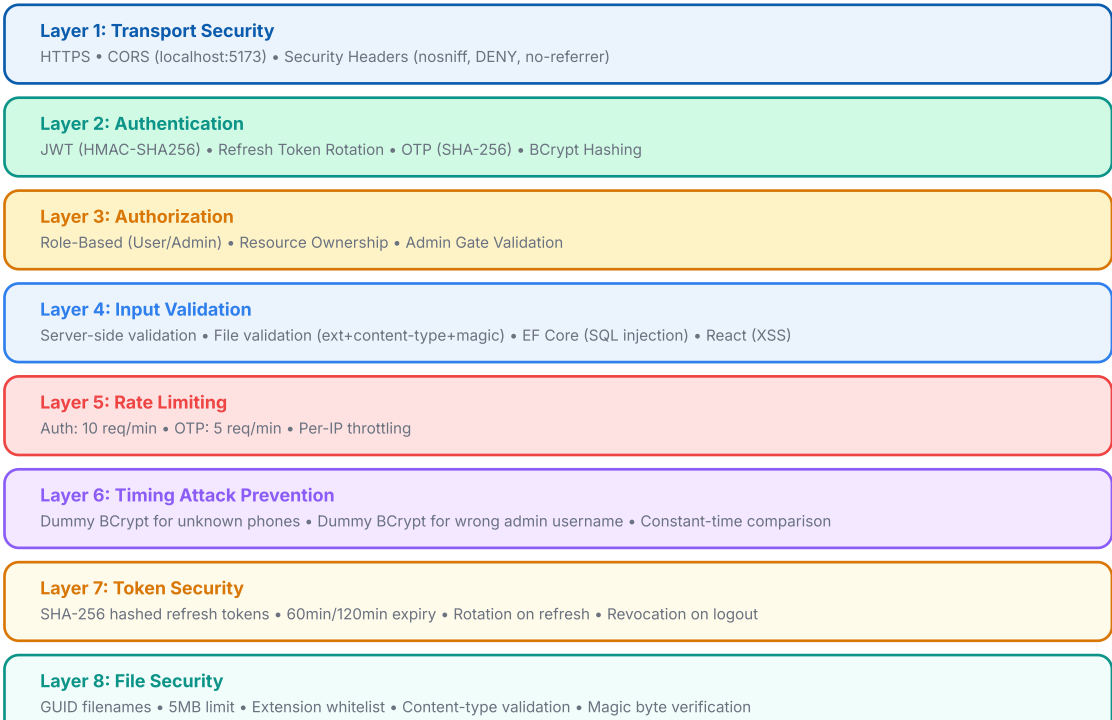


Figure 19: Security Architecture — Eight-Layer Defense

Threat Mitigation

Threat	Mitigation	Layer
Brute Force	Rate limiting + BCrypt cost	5, 2
Credential Stuffing	Rate limiting	5
Token Theft	Short expiry + rotation	7
Path Traversal	GUID filenames	8
SQL Injection	EF Core ORM	4
XSS	React auto-escaping	4
Timing Attacks	Dummy BCrypt	6
File Upload Attacks	Multi-layer validation	8
CORS Bypass	Origin whitelist	1
Clickjacking	X-Frame-Options: DENY	1

27 — Error Handling

The backend implements a global exception handling strategy through `ExceptionHandlerMiddleware` that converts all unhandled exceptions into standardized RFC 7807 ProblemDetails responses.

Exception Mapping

Exception Type	HTTP Status	Response
<code>InvalidOperationException</code>	409 Conflict	ProblemDetails with error message
<code>UnauthorizedAccessException</code>	403 Forbidden	ProblemDetails
All other exceptions	500 Internal Server Error	Sanitized ProblemDetails (no stack trace in production)

ServiceResult Pattern

All service methods return `ServiceResult` or `ServiceResult<T>` instead of throwing exceptions. Controllers inspect the `Success` property and map failures to appropriate HTTP status codes:

```
ServiceResult { bool Success; string? Error; }
ServiceResult<T> : ServiceResult { T? Value; }
```

28 — Validation

Input validation is enforced at multiple layers to ensure data integrity and prevent malicious input.

Server-Side Validation

- **Data Annotations:** Entity properties use `[Required]`, `[MaxLength]`, `[Precision]` attributes enforced by EF Core.
- **EF Configurations:** Fluent API configurations define unique indexes (PhoneNumber, EmailAddress, TokenHash, RentalRequestId), precision for decimal fields, and delete behaviors.
- **Service Layer:** Business rules are validated before persistence (e.g., category existence, listing ownership, overlap detection, OTP verification).

File Upload Validation

- **Extension Whitelist:** .jpg, .jpeg, .png, .gif, .webp only.
- **Content-Type Check:** MIME type must match expected image types.
- **Magic Byte Verification:** File header bytes are validated against known image signatures.
- **Size Limit:** Maximum 5MB per file, maximum 6 files per upload.
- **GUID Filenames:** User-provided filenames are replaced with GUIDs to prevent path traversal.

Client-Side Validation

- **React:** Form validation in Auth.tsx and ListingForm.tsx with real-time feedback.
- **Flutter:** Form validation in RegisterScreen and ListingFormScreen with translated error messages.
- **SQL Injection:** Prevented by Entity Framework Core's parameterized queries.
- **XSS:** Prevented by React's automatic HTML escaping and Flutter's widget-based rendering.

29 — Performance

Frontend Performance

- **Lazy Loading:** All pages are loaded via `React.lazy()` and `Suspense`, reducing initial bundle size.
- **Vite:** Fast HMR in development and optimized production builds with tree-shaking.
- **Image Optimization:** Listing images are served as static files from `wwwroot/uploads/` with browser caching.
- **Provider-Based State:** Minimal re-rendering through targeted Context providers.

Backend Performance

- **Scoped Services:** Per-request DI lifetime prevents state leakage and enables efficient resource management.
- **Repository Pattern:** Abstracts EF Core queries and enables future query optimization.
- **Pagination:** Browse endpoint limits results to 50 per page to prevent large payload transfers.
- **Static File Serving:** Images served directly by Kestrel without going through the application pipeline.

Database Performance

- **Unique Indexes:** On PhoneNumber, EmailAddress, TokenHash, and RentalRequestId for $O(\log n)$ lookups.
- **Appropriate Delete Behaviors:** CASCADE for owned data, RESTRICT to prevent accidental cascades.

- **Decimal Precision:** `Precision(12,2)` on all cost fields for efficient storage.

30 — Scalability

The current architecture is designed for single-server deployment. The following areas have been identified for future scalability improvements:

Current Limitations

- **Single Server:** API, database, and file storage run on a single machine.
- **In-Memory OTP:** OTP codes are stored in a `ConcurrentDictionary`, which does not survive restarts or scale across instances.
- **Local File Storage:** Uploaded images are stored on the local filesystem under `wwwroot/uploads/`.
- **No Caching Layer:** No Redis or distributed cache is implemented.

Scaling Path

- **Horizontal Scaling:** The stateless API (except OTP) can be load-balanced behind a reverse proxy.
- **Distributed Cache:** Redis could replace the in-memory OTP store and add response caching.
- **Cloud File Storage:** Azure Blob Storage or AWS S3 could replace local file storage for images.
- **Database Scaling:** Read replicas for browse-heavy workloads, connection pooling via EF Core.
- **Containerization:** Dockerfiles for API and frontend would enable consistent deployment across environments.

31 — Testing

Current State

The project has been validated through the following methods:

- **Backend Build Verification:** `dotnet build` compiles successfully with no errors across all four projects.
- **Frontend Build Verification:** `npm run build` produces a production-ready bundle.
- **Flutter Static Analysis:** `flutter analyze` reports "No issues found" across all 22 source files.
- **Manual Testing:** Core workflows (registration, login, listing creation, booking, rating) have been manually tested via Swagger UI and the web frontend.

Testing Gaps

- No automated unit tests are implemented.
- No integration tests for API endpoints.
- No end-to-end tests for web or mobile flows.
- No load testing or performance benchmarks.

Recommended Testing Strategy

- **Unit Tests:** xUnit for service layer logic, mock repositories.
- **Integration Tests:** WebApplicationFactory for API endpoint testing.
- **E2E Tests:** Playwright for web, `integration_test` for Flutter.
- **Load Tests:** k6 or Artillery for API performance under concurrent load.

32 — Deployment

Development Environment

Component	Value
Backend	<code>http://localhost:5000</code> (ASP.NET Core, HTTP)
Frontend	<code>http://localhost:5173</code> (Vite dev server)
Database	<code>localhost:1433</code> (SQL Server 2022, Docker)
Docker Container	<code>equipify-sql</code>
DB Credentials	<code>sa</code> / <code>Abs@12345</code>
Database Name	<code>EquipifyDb</code>
OTP Code (Dev)	<code>0000</code> (fixed)
Admin Login	<code>admin</code> / <code>Admin@12345</code>
Test User	<code>0790666835</code> / <code>123456</code>

CORS Configuration

Origins: `http://localhost:5173`, `http://127.0.0.1:5173`
Methods: GET, POST, PUT, DELETE
Headers: Content-Type, Authorization

Rate Limiting

Policy	Limit
<code>auth</code>	10 requests per minute
<code>otp</code>	5 requests per minute

Logging

Serilog is configured for console output and rolling file storage at `logs/equipify-api-.log` with Information, Warning, and Error levels.

33 — Future Improvements

The following enhancements have been identified for future development iterations:

High Priority

- **Payment Integration:** Implement Stripe or PayTabs for secure online payments and escrow-based rental transactions.
- **Push Notifications:** Firebase Cloud Messaging for real-time notifications on rental status changes.
- **Unit Testing Suite:** xUnit tests for service layer, integration tests for API endpoints.

Medium Priority

- **Real-Time Chat:** SignalR-based messaging between renters and owners.
- **Email Notifications:** Enable the existing EmailService for booking confirmations, status updates, and welcome emails.
- **Advanced Search:** Elasticsearch integration for full-text search with typo tolerance and ranking.
- **Image Optimization:** Automatic thumbnail generation and WebP conversion on upload.

Low Priority

- **Social Authentication:** Google, Facebook, and Apple OAuth integration.

- **Equipment Delivery:** Integration with logistics providers for delivery tracking.
- **Analytics Dashboard:** Advanced analytics with charts, trends, and exportable reports.
- **CI/CD Pipeline:** GitHub Actions for automated build, test, and deployment.
- **Docker Compose:** Full-stack development environment with a single `docker-compose up` command.

34 — Conclusion

Equipify successfully demonstrates the design and implementation of a multi-platform equipment rental marketplace that addresses the challenges of fragmented discovery, trust deficits, and manual scheduling in the equipment rental industry.

The system delivers a cohesive experience across three platforms — a React web application, a Flutter mobile application, and an ASP.NET Core backend API — unified by a shared RESTful API, consistent data models, and a bilingual (Arabic/English) user interface.

Key achievements of the project include:

- A secure authentication system with phone-based OTP verification, JWT tokens, and refresh token rotation.
- A complete rental lifecycle with OTP-verified booking, overlap detection, owner acceptance, and post-rental rating.
- An admin moderation layer ensuring marketplace quality through listing approval and user management.
- A bilingual architecture supporting Arabic and English with full RTL layout switching.
- An eight-layer security architecture addressing common web application threats.
- A Clean Architecture backend with 48 API endpoints, 9 services, and a generic repository pattern.

The project establishes a solid foundation for future enhancements including payment integration, real-time messaging, push notifications, and advanced search capabilities. The modular architecture ensures that these additions can be implemented with minimal disruption to existing functionality.

Appendix A — Glossary

Term	Definition
Listing	An equipment item posted by an owner for rent, including images, pricing, location, and availability.
Rental Request	A booking submission from a renter for a specific listing, with date/time range and OTP verification.
OTP	One-Time Password — a temporary code sent to a phone number for verification purposes.
JWT	JSON Web Token — a compact, URL-safe token for securely transmitting information between parties.
Refresh Token	A long-lived token used to obtain new JWT access tokens without requiring re-authentication.
BCrypt	A password hashing function designed to be slow and computationally expensive, resisting brute-force attacks.
Rental Unit	The time-based pricing unit for a listing: hour, day, week, month, or year.
Admin Moderation	The process by which administrators review and approve or reject listing submissions before they become publicly visible.
ServiceResult	A response pattern used in the service layer that wraps success/error status and optional data, avoiding exception-based control flow.
Clean Architecture	An architectural pattern that organizes code into concentric layers (Domain, Data, Service, Api) with dependencies pointing inward.

Appendix B — Abbreviations

Abbreviation	Full Form
API	Application Programming Interface
AR	Arabic

Abbreviation	Full Form
CRUD	Create, Read, Update, Delete
CORS	Cross-Origin Resource Sharing
DB / Db	Database
DFD	Data Flow Diagram
DI	Dependency Injection
EF Core	Entity Framework Core
EN	English
ER	Entity-Relationship
FK	Foreign Key
HMAC	Hash-based Message Authentication Code
JWT	JSON Web Token
LTR	Left-to-Right
ORM	Object-Relational Mapping
OTP	One-Time Password
PK	Primary Key
REST	Representational State Transfer
RTL	Right-to-Left
SHA	Secure Hash Algorithm
SQL	Structured Query Language
UML	Unified Modeling Language
UUID / GUID	Universally Unique Identifier / Globally Unique Identifier

